FAST- National University of Computer and Emerging Sciences, Karachi.

FAST School of Computing

Assignment # 2 (CLO-2), Spring 2025

CS3001- Computer Networks

ASSIGNMENT-II (RELIABLE DATA TRANSFER)

Submission Guidelines:

- **Due date: Sunday, 13th April, 2025.**
- **This is a programming assignment and the submission will be based on a 4-5 minute demo followed by a short viva.**
- **You must submit the code and screenshot of the working demo on Google classroom.**
- **You must mark the attendance during the demo. The student ID, name and section must be mentioned.**
- **Plagiarism in any form will result in straight "F".**

Objective:

The goal of this assignment is to implement the Reliable Data Transfer Protocol (rdt 3.0) and extend it to include Go-Back-N (GBN) and Selective Repeat (SR) protocols. You will simulate a uni-directional data transfer where control information flows in both directions.

Requirements:

1. **Protocol Specification**:
    - Implement the rdt 3.0 protocol.
    - Extend the implementation to include Go-Back-N (GBN) and Selective Repeat (SR) protocols.
    - Use Finite State Machines (FSMs) to model the sender and receiver for each protocol.
    - Ensure the protocols handle packet loss, corruption, and delays.
2. **Functionality**:
    - **rdt 3.0 (Stop-and-Wait)**:
        - Sender sends one packet at a time and waits for an acknowledgment (ACK) before sending the next packet.
        - Implements a timer to handle packet loss.
        - Retransmits the packet if an ACK is not received within a specified timeout period.
        - Receiver sends ACKs for correctly received packets.
    - **Go-Back-N (GBN)**:
        - Sender can have up to N unacknowledged packets in the pipeline.
        - Receiver only accepts packets in order and discards out-of-order packets.
        - Sender retransmits all packets starting from the oldest unacknowledged packet on a timeout.

- **Selective Repeat (SR)**:
    - Sender can have up to N unacknowledged packets in the pipeline.
    - Receiver accepts out-of-order packets and buffers them until all previous packets are received.
    - Sender retransmits only the lost or corrupted packets.

3. **Implementation Details**:
    - Use any programming language of your choice (e.g., Python, Java, C++).
    - Simulate the unreliable network by introducing random packet loss, corruption, and delays.
    - Implement the FSM for both sender and receiver for each protocol.
    - Use sequence numbers to ensure in-order delivery.
    - Make the size and count of packets to be transferred configurable.
    - Assume a fixed timeout period for retransmissions.
4. **Testing**:
    - Test your implementation with different scenarios:
        - No packet loss or corruption.
        - Packet loss.
        - Packet corruption.
        - Delayed packets.
    - Ensure that the protocols recover from errors and deliver all packets correctly.
5. **Documentation**:
    - Provide a brief report explaining your implementation.
    - Include the FSM diagrams for the sender and receiver for each protocol.
    - Describe the testing scenarios and results.

Evaluation Criteria:

- Correct implementation of rdt 3.0, GBN, and SR protocols.
- Proper handling of packet loss, corruption, and delays.
- Correct use of sequence numbers and acknowledgments.
- Quality and clarity of the documentation.

Note:

- You need to implement the **Stop-and-Wait protocol** as part of the **rdt 3.0** implementation, where only a single packet is in-flight.
- The size and count of packets to be transferred should be configurable.
- You can assume a fixed timeout period for retransmissions.